

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет

Механико-математический

Кафедра
Направление подготовки

Программирования
Математика и компьютерные науки

РЕФЕРАТ

Новиков Сергей Александрович

(Фамилия, Имя, Отчество автора)

Тема:

Обзор средств записи и воспроизведения исполнения многопоточных программ с помощью средств операционной системы

«Реферат принят»

Научный руководитель

Магистр,
индустриальный доцент
кафедры программирования ММФ

Филатов А.Ю. / _____
(ФИО, Подпись)

«...» 20... г.

Новосибирск, 2025

Содержание

1	Введение	3
2	Описание системы	3
2.1	Record-and-Replay: способ записи и воспроизведения	3
2.2	Ограничения многопоточного исполнения	4
3	Технические детали реализации	5
3.1	Ptrace	5
3.2	Контроль системных вызовов	5
3.3	Контроль асинхронных событий	7
3.3.1	”Внутренние часы”	7
3.3.2	Ограничения счетчиков	8
4	Техники ускорения и оптимизации	9
4.1	Снижение места под данные журналов	9
4.2	In-process system call interception	10
5	Путешествие во времени	10
6	Выводы	11

1 Введение

Отладка программного обеспечения является неотъемлемой и зачастую наиболее трудоемкой частью процесса разработки. На заре появления такой возможности, традиционные отладчики, такие как GDB¹ или LLDB², имели фундаментальное ограничение: возможность двигаться по коду преимущественно в одном направлении – вперед³. Это существенно затрудняет поиск причин сложных, особенно недетерминированных или редко воспроизводимых ошибок, когда точные условия возникновения сбоя трудно или невозможно воссоздать вручную.

Так появилась концепция Записи и Воспроизведения. Суть её заключается в фиксации всех недетерминированных входных данных и событий (например, системных вызовов, обработчиков сигналов, событий вытеснения потоков планировщиком ОС) во время выполнения программы таким образом, чтобы это выполнение можно было впоследствии точно воспроизвести. Инструмент, реализующий Записи и Воспроизведение, позволяет многократно "проигрывать" проблемный запуск программы, что значительно облегчает тестирование и выявление ошибок, гарантируя их стопроцентную повторяемость в рамках записанного сеанса.

Тем не менее, возможности простого воспроизведения не всегда достаточны для глубокого понимания причин сбоя. Современные требования к отладке диктуют необходимость в более мощных инструментах. Разработчики стремятся не просто повторить ошибку, а получить возможность "путешествовать во времени" по истории выполнения программы: выполнять шаги назад (reverse debugging), устанавливать точки останова на прошлые события и исследовать состояние программы в любой произвольный момент времени (time travel debugging). Эти возможности позволяют быстро локализовать источник ошибки, отслеживая, как некорректное состояние программы возникло и распространялось.

Одним из наиболее ярких и эффективных инструментов, реализующих эти продвинутые возможности на базе механизма записи и воспроизведения на ОС Linux x86/x86_64, является отладчик rr (Record and Replay debugger), изначально разработанный в Mozilla. В данном реферате мы подробно рассмотрим подходы и технические решения, предложенные инженерами проекта rr (далее – RR) в их статье [1] для создания этого отладочного инструмента.

2 Описание системы

Система RR состоит из двух взаимодополняющих компонентов — механизмов записи (Record) и воспроизведения (Replay). Поговорим подробнее про каждый из этих компонентов.

2.1 Record-and-Replay: способ записи и воспроизведения

- **Концепция Record**

Когда RR запускает программу в режиме записи, то он делает следующее:

1. **Наблюдение за взаимодействием с внешним миром:**

Программа постоянно взаимодействует с операционной системой: читает файлы, отправляет данные по сети, получает текущее время, реагирует на ввод пользователя и т.д. Эти взаимодействия могут давать разные результаты при каждом запуске. RR тщательно следит за каждым таким обращением программы к "внешнему миру" (к ядру ОС) и аккуратно записывает:

- Что программа попросила? (Например, "прочитать 100 байт из файла X").
- Что ей ответил внешний мир? (Например, "вот эти 100 байт" или "ошибка, файл не найден"). Все эти ответы и данные записываются.

2. **Фиксация "поворотных моментов" во времени:**

В программе могут происходить события, время которых заранее неизвестно:

- **Сигналы:** Внезапные уведомления от операционной системы как пришедшие извне, например `kill -SIGINT <PID>`, так и возникшие внутри процесса, например при обращении к неактивным страницам виртуального адресного пространства (page fault). RR записывает, какой именно сигнал пришел и в какой момент выполнения программы это случилось.

¹<https://sourceware.org/gdb/>

²<https://lldb.llvm.org/>

³Поправка: на текущий момент (2025 год) и GDB, и LLDB могут двигаться и вперед, и назад, но мы будем говорить о том, что они умели в прошлом.

– **Работа с потоками (threads):**

Если программа многопоточная, порядок, в котором потоки получают процессорное время и взаимодействуют друг с другом (через общую память, синхронизацию и т.д.), может меняться от запуска к запуску. RR старается упорядочить эту работу или, как минимум, очень точно зафиксировать, какой поток сколько "поработал", прежде чем управление перешло к другому потоку или произошло какое-то внешнее событие. Он записывает эту последовательность переключений и "объем работы", выполненный каждым потоком между этими переключениями.

– **Отслеживание некоторых внутренних непредсказуемостей:**

Некоторые процессорные инструкции могут давать разные результаты (например, генерация случайных чисел). RR также старается зафиксировать результаты таких операций.

В итоге получается некий "журнал" всех внешних воздействий и внутренних непредсказуемых событий, который можно использовать для воспроизведения программы.

• **Концепция Replay**

Теперь, имея этот детальный журнал, RR может запустить программу снова, но уже в совершенно других условиях:

1. **Изоляция от реального внешнего мира:**

Когда программа во время воспроизведения пытается снова обратиться к операционной системе (прочитать файл, получить время и т.д.), RR перехватывает этот запрос. Вместо того чтобы обращаться к реальной ОС, RR подменяет результат системного вызова данными, которые были сохранены в журнал. Программа "думает", что общается с реальным миром, но на самом деле получает ответы из прошлого.

2. **Воссоздание последовательности событий:**

- **Сигналы:** RR следит за ходом выполнения программы. Когда программа доходит до того "момента", когда в оригинальном запуске пришел сигнал, RR искусственно доставляет ей точно такой же сигнал.
- **Работа с потоками:** RR использует записанную информацию о переключениях и "объеме работы" потоков, чтобы заставить их выполняться в точно такой же последовательности и с такими же интервалами, как это было при записи. Никакой "случайности" в планировании потоков больше нет.

3. **Повторение внутренних непредсказуемостей:**

Если программа обращается к инструкции, результат которой был записан, RR предоставляет ей этот сохраненный результат.

В итоге программа выполняется **абсолютно идентично** тому, как она выполнялась во время записи.

2.2 Ограничения многопоточного исполнения

Многопоточные ограничения при записи:

Когда RR записывает многопоточное приложение, он, по сути, берет на себя значительную часть функций планировщика ОС, но только для потоков целевого процесса.

- **Снижение истинного параллелизма:** Вместо того чтобы позволить ядру ОС свободно планировать потоки приложения по доступным CPU-ядрам, RR обычно **ограничивает их выполнение одним ядром** и сам решает, какой поток будет выполняться следующим. Эмуляция работы на одном ядре означает, что программа во время записи не использует всю мощь многоядерного процессора.
- **Выбор конкретного сценария планирования:** В стандартном режиме записи и воспроизведения RR записывает один конкретный сценарий взаимодействия потоков, который случился во время сессии записи. Он не пытается исследовать все возможные сценарии.⁴

Многопоточные ограничения при воспроизведении:

Во время воспроизведения RR использует свой журнал, чтобы заставить потоки вести себя идентично.

⁴С 2016 года RR представил режим воспроизведения записанной программы с возможностью исследования большого количества сценариев, основанный на рандомизации приоритетов потоков (chaos mode). В данной статье мы будем рассматривать только стандартный режим

- **Воспроизводится только записанный сценарий:** Если ошибка синхронизации (например, гонка данных) не проявилась во время конкретной сессии записи (из-за того, как RR упорядочил потоки), она не проявится и при воспроизведении этой записи. Для ее поимки может потребоваться несколько попыток записи или специфические условия.

В целом, RR для многопоточности создает управляемую, детерминированную среду, фиксируя одно из возможных поведений системы.

3 Технические детали реализации

Реализация RR ориентирована на минимальное вмешательство в работу целевого приложения и окружения, используя исключительно возможности, предоставляемые современными процессорами и ОС. Рассмотрим основные инженерные решения, которые позволили этого достичь.

3.1 Ptrace

Ptrace⁵ (**process trace**) — системный вызов в Linux, который предоставляет одному процессу (трассировщику, **tracer**) возможность контролировать и наблюдать за выполнением другого процесса (трассируемого, **tracee**). Это ключевое средство, за счет которого достигается большинство возможностей RR.

В его реализации нам важно:

1. Трассировщик использует `ptrace` с определенным запросом (например, `PTTRACE_ATTACH` или `PTTRACE_TRACEME` при запуске), чтобы "прикрепиться" к трассируемому процессу. С этого момента ядро знает, что этот процесс находится под внешним контролем.
2. Когда в трассируемом процессе происходит "интересное" событие (например, он входит в системный вызов, собирается получить сигнал, выполняет инструкцию точки останова), ядро **приостанавливает** выполнение трассируемого процесса. Затем ядро **уведомляет** трассировщика об этом событии, посылая ему сигнал (обычно, `SIGTRAP`).
3. Если трассируемый процесс остановлен, трассировщик может использовать другие запросы к `ptrace` для **чтения/записи** регистров CPU трассируемого процесса или его памяти, а также для **управления его дальнейшим выполнением** (например, продолжить нормально, выполнить одну инструкцию, пропустить текущий системный вызов, отправить или перехватить сигнал и т.д.).

Именно за счет этого системного вызова RR способен перехватывать системные вызовы, обрабатывать сигналы, управлять выполнением и получать доступ к памяти и состоянию отслеживаемой программы.

Тем не менее, `ptrace` имеет свои ограничения и проблемы:

- Каждое событие, перехватываемое `ptrace` (например, вход/выход из системного вызова, сигнал), требует как минимум двух переключений контекста: от трассируемого процесса к ядру, затем от ядра к трассировщику, и обратно. Это значительно замедляет выполнение трассируемого процесса, особенно если системных вызовов много.
- Некоторые системные вызовы могут быть сложны для точного перехвата и эмуляции всех их побочных эффектов, особенно если они зависят от внутреннего состояния ядра, информация о котором недоступна через `ptrace`.
- Хотя концепция `ptrace` есть во многих UNIX-подобных системах, конкретная реализация, доступные запросы и их поведение могут немного отличаться, что усложняет создание полностью кроссплатформенных отладчиков, опирающихся только на `ptrace`.

3.2 Контроль системных вызовов

Для RR, точный контроль системных вызовов — это фундамент, на котором строится вся система записи и воспроизведения. Ведь именно через системные вызовы программа взаимодействует с недетерминированным внешним миром (файловая система, сеть, ядро, другие процессы).

RR использует `ptrace` (в частности, его возможность останавливать процесс на входе и выходе из системного вызова через `PTTRACE_SYSCALL`) для реализации подобного контроля. В качестве примера, рассмотрим системный вызов `read` на рисунке 1 (на нем отображена фаза Записи):

⁵<https://man7.org/linux/man-pages/man2/ptrace.2.html>

1. Фаза Записи (Record):

- **Остановка перед вызовом:** Когда трассируемый процесс собирается выполнить системный вызов `sys_read`, `ptrace` уведомляет RR, выступающим в роли трассировщика (Tracer Thread). Трассируемый процесс останавливается перед тем, как ядро начнет обрабатывать вызов.

В этот момент RR до срабатывания уведомления узнает об остановке через `wait_pid(T)`⁶, считывает номер системного вызова и все его аргументы (из регистров, из памяти по указателям), выполняет **redirect arg2 to scratch**. Это важный момент: **arg2** для `read` – это указатель на буфер, куда будут читаться данные. RR может подменить этот указатель на свой собственный "черновой" (`scratch`) буфер. Это дает RR полный контроль над тем, куда ядро запишет данные, и позволяет избежать проблем с доступом к памяти трассируемого процесса или изменением этой памяти до того, как RR ее залогировает.

Затем RR разрешает трассируемому процессу продолжить выполнение и передать управление непосредственно в системный вызов, с помощью `ptrace(T, CONT_SYSCALL)`.

- **Остановка после вызова:** После того как ядро завершило обработку системного вызова, но до того, как управление вернулось обратно в пользовательский код трассируемого процесса, `ptrace` снова останавливает трассируемый процесс и уведомляет RR.

RR снова ждет `wait_pid(T)`. Он получает результат `N = syscall_result_reg` (количество прочитанных байт). Затем **save N scratch bytes to trace** – эти N байт из "чернового" буфера сохраняются в журнал. И очень важный шаг: **copy N scratch bytes to buf** – RR копирует данные из своего "чернового" буфера в оригинальный буфер трассируемого процесса. Теперь для процесса все выглядит так, как будто `read` выполнялся совершенно нормально и записал данные в его буфер.

RR разрешает трассируемому процессу продолжить обычное выполнение, используя `ptrace(T, CONT_SYSCALL)`.

2. Фаза Воспроизведения (Replay):

Когда программа при воспроизведении доходит до места, где ранее был системный вызов, RR (снова через `ptrace`) перехватывает эту попытку. **Ключевой момент:** RR не позволяет ядру выполнить этот системный вызов повторно.

Вместо этого RR обращается к своему логу (`trace`), находит запись об этом конкретном экземпляре системного вызова, **подменяет** результат, устанавливая регистр возвращаемого значения (например, `rax`) тем значением, которое было записано в лог. Если системный вызов при записи изменял память (как `read`), RR копирует записанные ранее данные из лога в соответствующие области памяти трассируемого процесса.

Программа **продолжает** выполнение, "думая", что системный вызов только что успешно (или неуспешно, как было записано) завершился.

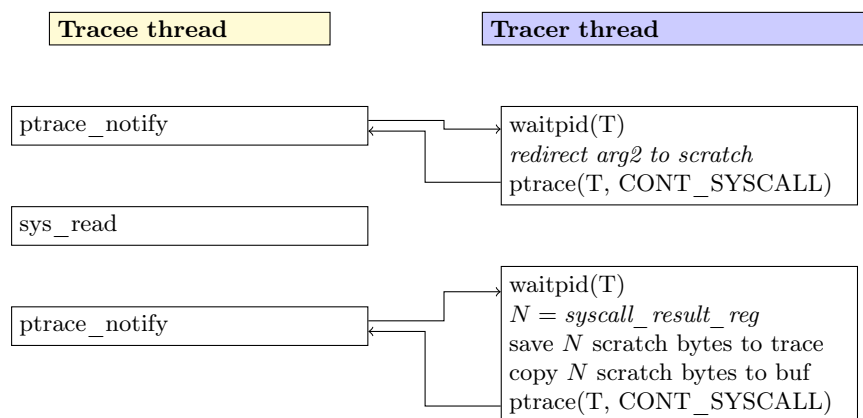


Рисунок 1: Процесс записи системного вызова

Есть еще нюансы, связанные с системными вызовами:

⁶<https://man7.org/linux/man-pages/man3/wait.3p.html>

1. Системные вызовы могут быть определены **слишком произвольно**⁷ (например, `ioctl`⁸). RR должен уметь для распространенных реализаций `ioctl` определять, какие области памяти являются входными, какие выходными, и записывать/воспроизводить их содержимое.
2. Некоторые системные вызовы взаимодействуют с потоками или памятью (например, `mmap`⁹ и `execve`¹⁰):
 - **mmap**: изменяет адресное пространство процесса. RR должен точно записать параметры `mmap` (адрес, размер, права доступа, флаги, дескриптор файла, смещение) и его результат (полученный адрес). При воспроизведении он должен эмулировать этот `mmap`, чтобы адресное пространство было идентичным (в частности, размещение страницы на запомненный адрес достигается с помощью флага `MAP_FIXED`).
 - **execve**: создает новую программу, с новым стеком, кучей, и сегментами данных. RR должен это зафиксировать, записать аргументы, окружение нового процесса и повторить то же поведение `execve`, что и во время записи, загружая те же данные.

3.3 Контроль асинхронных событий

Когда программа выполняется, на нее могут влиять события, происходящие в непредсказуемые моменты – асинхронные события. Ключевыми из них для RR являются системные сигналы и решения операционной системы о переключении между потоками (смена контекста). Чтобы отладчик мог точно воспроизвести выполнение, он должен не только знать, что произошло, но и когда именно это произошло с точки зрения самой программы.

3.3.1 "Внутренние часы"

Асинхронный сигнал может прервать исполнение программы в произвольный момент времени. Если мы просто запишем "пришел сигнал X", этого будет недостаточно. При повторном запуске тот же сигнал может прийти чуть раньше или чуть позже, застав программу в другом состоянии (с другими значениями в регистрах, на другой инструкции), и ошибка либо не воспроизведется, либо проявится иначе.

Поскольку сигнал может прервать исполнение программы в произвольный момент времени, непонятно, как измерить эту самую "точку выполнения". Нужны **"внутренние часы"** — логический механизм времени, однозначно связывающий данный сигнал с моментом времени (совокупностью определенных данных) в программе, когда программа начала этот сигнал обрабатывать.

Для реализации подобных "внутренних часов" инженерами Mozilla [1] были взяты счетчики производительности (HPC¹¹) на базе современных процессоров Intel. Это специальные регистры процессора, которые аппаратно считают различные события, происходящие во время исполнения программы. Среди них:

- **Instructions retired**: количество выполненных инструкций. Данная величина, в зависимости от настроек процессора, может учитывать спекулятивное исполнение (исполнение процессором инструкций, результат которых может быть проигнорирован), или игнорировать его.
- **Retired conditional branches**: количество выполненных условных переходов. Условным переходом считается инструкция, которая передает управление по другому адресу (на метку) программы только если выполняется определенное условие (например, такими инструкциями являются `je`, `j1`, `jz` и т.д.). Данная величина также может учитывать спекулятивное исполнение условных ветвлений.

Определим, что аппаратный счетчик производительности **детерминирован**, если после каждого запуска одной и той же программы каждое его значение совпадает с соответствующими значениями всех других запусков.

В контексте данного определения мы будем рассматривать счетчики, не учитывающие спекулятивное исполнение, потому что поведение таких счетчиков с данным режимом трудно контролировать.

⁷Системный вызов выполняется **слишком произвольно** означает, что его поведение полностью зависит от переданных ему аргументов, в частности от переданного кода команды и указателя на данные, то есть в зависимости от этих аргументов реализации данного системного вызова могут кардинально отличаться.

⁸<https://man7.org/linux/man-pages/man2/ioctl.2.html>

⁹<https://man7.org/linux/man-pages/man2/mmap.2.html>

¹⁰<https://man7.org/linux/man-pages/man2/execve.2.html>

¹¹https://en.wikipedia.org/wiki/Hardware_performance_counter

В статье [2] в качестве одной из причин недетерминированности счетчиков рассматривались **прерывания**. Когда прерывание происходит, процессор сохраняет текущее состояние программы (в том числе инструкцию, на которой он останавливается), переходит в режим ядра и начинает выполнять обработчик прерывания. После обработки прерывания, когда управление передается обратно в программу, процессор восстанавливает предыдущее состояние программы и продолжает выполнение. В зависимости от ситуации (состояния ядра ОС или по иным причинам), инструкция, с которой начнется исполнение после восстановления, может быть либо той же инструкцией, на которой было прервано исполнение, или следующей. Это может привести к тому, что счетчик производительности будет увеличиваться в разных запусках программы по-разному, даже если программа выполняется идентично. Такое происходит, например, при генерации прерывания, следующего за возникновением **Page Fault**.

Важно понимать, что мы говорим не про прерывания, поведение которых RR записывает и воспроизводит¹², а про прерывания, которые возникают во время самого воспроизведения программы, и к которым RR не имеет никакого отношения. Самый очевидный пример — это прерывания от аппаратных таймеров, которые обрабатывает сам планировщик ОС. Так как RR является таким же процессом, как и все остальные, он также может быть прерван планировщиком в любой момент времени.

Исследование [2] показало, что в современных процессорах на базе Intel счетчик **Retired Conditional Branches** является детерминированным, в то время как счетчик **Instructions Retired** таковым не является, как минимум потому что подвержен влиянию прерываний. Поэтому RR использует именно счетчик RCB для определения момента времени во "внутренних часах".

В режиме Записи, когда RR перехватывает сигнал, он немедленно считывает текущее значение этих "внутренних часов" и записывает в журнал тип сигнала, показание "внутренних часов" в этот момент, а также полное состояние регистров CPU и, возможно, ключевые участки стека¹³ (далее станет ясно, почему это важно).

В режиме Воспроизведения RR также отслеживает показания "внутренних часов". Как только они достигают значения, которое было записано в логах для данного сигнала, RR останавливает программу, восстанавливает записанное состояние регистров/стека и "доставляет" программе тот же самый сигнал. Это гарантирует, что сигнал застанет программу в абсолютно идентичном состоянии.

3.3.2 Ограничения счетчиков

Как уже было сказано, помимо значения "внутренних часов", RR также записывает полное состояние регистров и иногда даже состояние стека. Для того, чтобы понять, зачем так нужно, рассмотрим следующие примеры кода:

```
1  loop:
2      mov eax, 1
3      mov ebx, 2
4      inc eax
5      jmp loop
```

Листинг 1: Ассемблерный код с необходимостью записи регистров

¹²Общая идея контроля записи и воспроизведения прерываний не сильно отличается от контроля асинхронных событий, поэтому мы не станем рассматривать их отдельно.

¹³Уточнение про сохранение "состояние стека": По большей части под ним подразумевается сохранение значения регистров **rsp** и **rbp** (они определяют текущий кадр стека) и последовательности сохраненных регистров **rbp** (это адреса на фреймы предыдущих функций, которые помогают определить стек вызовов).


```

1  bar():
2      push    rbp
3      mov     rbp, rsp
4      mov     DWORD PTR [rbp-4], 1    ; int a = 1;
5      pop     rbp
6      ret
7  foo():
8      push    rbp
9      mov     rbp, rsp
10     call    bar()
11     call    bar()
12     call    bar()
13     pop     rbp
14     ret

```

Листинг 2: Ассемблерный код с необходимостью записи регистров и стека

```

1  loop:
2      inc     [address]
3      jmp     loop

```

Листинг 3: Ассемблерный код с проблемой

Уже в листинге 1 легко понять, что одних "внутренних часов" не достаточно для определения конкретной инструкции, поскольку здесь нет условных ветвлений и данный счетчик вообще не изменится. Тем не менее, меняется состояние регистров, которое нужно сохранить. Логично предположить, что одного регистра IP, хранящего адрес смещения следующей инструкции, будет достаточно. Но это не так, потому что в качестве контрпримера достаточно рассмотреть любой цикл, который проходит несколько раз некоторую инструкцию (например, `add eax, 1`). В каждый момент прохождения в цикле данной инструкции регистр IP будет указывать на нее (так как адрес в памяти тот же), а счетчик RCB не изменится (так как инструкция `jmp` не является условным ветвлением). Однако, если бы мы сохранили состояние регистра `eax`, то смогли бы определить конкретную итерацию цикла, на которой нам нужно воспроизвести сигнал.

В листинге 2 также не происходит изменения "внутренних часов", так как переход в другую функцию (вызов `call` не является условным ветвлением). Помимо этого, сохранения регистров будет недостаточно, поскольку сведения лишь о регистрах и "внутренних часах" не позволят понять, какая функция была вызвана. Чтобы однозначно определить функцию среди набора идентичных, нужно хранить ключевые участки стека, потому что там гарантированно есть различная информация, по которой можно отличить вызовы функций. Так, в листинге 2 на стеке будет храниться адрес возврата из соответствующей функции `bar` в функцию `foo`, а именно на соответствующую `call` инструкцию, адрес смещения которой отличается от адреса двух других.

В листинге 3 не будет меняться ничего из выше перечисленного, и, следовательно, вызовет проблемы. Тем не менее, в статье [1] упоминается, что на практике в большинстве случаев подобный сценарий возникает очень редко, поскольку код, сгенерированный компилятором, почти всегда взаимодействует со стеком (как минимум в прологе и эпилоге, и этого уже в основном достаточно, чтобы однозначно определить конкретную функцию), однако если он возникнет, то RR просто не сможет воспроизвести его корректно.

4 Техники ускорения и оптимизации

4.1 Снижение места под данные журналов

Цель RR – записать ровно столько информации, сколько необходимо для точного детерминированного воспроизведения. Какие вопросы нужно решить:

- Между двумя событиями, которые RR должен зафиксировать (например, двумя системными вызовами или сигналом), программа может выполнить миллионы инструкций. Если эти

инструкции не взаимодействуют с внешним миром и их поведение полностью предсказуемо, то нет смысла записывать состояние программы после каждой из них. Это достигается за счет упомянутых выше **аппаратных счетчиков производительности (НРС)**, которые позволяют RR игнорировать большие участки кода, где не происходит никаких недетерминированных событий.

- Предположим, что есть системный вызов `read()`, который просит прочитать 1 мегабайт данных из файла. Если в файле осталось только 10 килобайт, то будет прочитано именно 10 КБ. Если бы RR всегда резервировал в журнале место под запрошенный 1 МБ, журнал бы быстро раздулся.

RR записывает в журнал только те данные, которые фактически были переданы или изменены системным вызовом. В нашем примере с `read()`, если было прочитано 10 КБ, то именно эти 10 КБ (и информация о том, что было прочитано 10 КБ) и попадут в журнал. Это значительно экономит место, особенно для программ с активным файловым или сетевым IO.

- Файлы журналов, создаваемые RR, часто автоматически сжимаются с использованием стандартных алгоритмов компрессии (например, с помощью `zlib` в deflate-формате).

4.2 In-process system call interception

Это техника, при которой RR встраивает небольшой код непосредственно в адресное пространство отлаживаемой программы (tracee) для перехвата системных вызовов.

В память трассируемого процесса инжектируется¹⁴ специальный "буфер системных вызовов" (`syscall buffer`) и небольшой код-обработчик (`stub`). Точки входа в системные вызовы (например, в `libc`¹⁵) модифицируются так, чтобы вызывать этот инжектированный `stub`. Через `syscall buffer` RR может взаимодействовать со `stub`, который в распоряжении в адресном пространстве трассируемого процесса.

В фазе Записи `Stub` записывает аргументы системного вызова в `syscall buffer`. Затем он может либо выполнить оригинальный системный вызов прямо внутри себя, либо сигнализировать RR (как трассировщику), который затем через `ptrace` даст команду на выполнение. После выполнения результат также записывается.

В фазе Воспроизведения `Stub` записывает "намерение" **вызвать `syscall`** в буфер. Основной процесс RR видит это, находит в журнале записанный ранее результат этого вызова и передает его обратно в `stub`. `Stub` эмулирует завершение `syscall`'а, подставляя записанные значения (возвращаемое значение, данные в буферах). Реальный системный вызов к ядру не происходит.

Из плюсов данной техники можно выделить:

- **Снижение накладных расходов:** Вызов системного вызова через `ptrace` требует переключения контекста между процессами, что значительно замедляет выполнение. Используя in-process interception, RR может избежать этого переключения, что приводит к значительному увеличению производительности.
- **Гибкость:** Дает больше контроля над исполнением внутри трассируемого процесса.

Из минусов можно отметить:

- **Сложность:** Инъекция кода, его синхронизация с внешним трассировщиком, обработка пограничных случаев (сигналы, многопоточность) — нетривиальные технические решения, потребовавшие бы значительного количества усилий на их надежную и эффективную реализацию.

5 Путешествие во времени

"Путешествие во времени" в RR позволяет разработчику установить состояние программы в любой момент, зафиксированный во время сессии записи. Для этого RR всегда начинает воспроизведение с самого начала записанной трассы и быстро "проматывает" исполнение, используя свой журнал событий и значения "внутренних часов", до указанной пользователем точки в прошлом. Достигнув цели, программа останавливается в состоянии, идентичном тому, что было при оригинальной записи, предоставляя полный доступ для анализа в отладчике.

¹⁴https://en.wikipedia.org/wiki/DLL_injection

¹⁵<https://www.gnu.org/software/libc/>

Концепция Запись и Воспроизведение, лежащая в основе RR, стала основой для реализации таких возможностей, как **Time-travel debugging** и **Reverse Debugging**.

Данные способы отладки являются натуральным продолжением RR. "Шаг вперед" — это просто воспроизведение до нужной точки, а "шаг назад" достигается не истинным обратным исполнением, а восстановлением состояния программы из ранее сохраненного **чекпоинта** (снимка памяти и регистров) и быстрым воспроизведением вперед с этого чекпоинта до точки, непосредственно предшествующей желаемому моменту отката.

6 Выводы

Система RR, основанная на принципе тщательной записи всех недетерминированных событий и их точной привязки к "внутренним часам" программы, а затем детерминированного воспроизведения этих событий из сохраненного журнала, предоставляет важные преимущества для процесса отладки. Самое значительное из них — это способность надежно воспроизводить даже самые неуловимые ошибки, которые меняют свое поведение или исчезают при попытке их отладки традиционными методами. Записав сбойное исполнение с помощью RR один раз, его можно воспроизводить сколько угодно раз, и оно будет проходить абсолютно идентично, что является краеугольным камнем для эффективного анализа.

Кроме того, поскольку основная работа по сбору данных происходит во время записи (с оптимизированными накладными расходами), а сама отладка выполняется на этапе воспроизведения, процесс интерактивного анализа оказывает минимальное влияние на поведение воспроизводимой программы. Это позволяет избежать искажений, которые могут вносить традиционные отладчики. Более того, после создания записи разработчик получает возможность многократно и с разных сторон анализировать один и тот же проблемный сценарий, используя различные техники отладки и не опасаясь, что условия изменятся и ошибка не воспроизведется снова.

Таким образом, RR коренным образом трансформирует процесс отладки сложных программных систем, особенно тех, где недетерминизм играет существенную роль, как, например, в многопоточных приложениях или системах с асинхронным вводом-выводом, предоставляя разработчикам беспрецедентный контроль и возможности для понимания первопричин ошибок.

Список литературы

- [1] R. O’Callahan, C. Jones, N. Froyd, K. Huey, A. Noll, and N. Partush. Engineering record and replay for deployability: Extended technical report. *CoRR*, abs/1705.05937, 2017.
- [2] V. Weaver, D. Terpstra, and S. Moore. Non-determinism and overcount on modern hardware performance counter implementations. *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*, pages 215–224, 04 2013.